

EKSAMENSOPPGAVE

Bokmål

Emnekode: DAT108

Emnenavn: Programmering og webapplikasjoner

Klasse: 2. klasse DATA / INF

Dato: 18. desember 2023

Eksamensform: Skriftlig skoleeksamen (Wiseflow | FLOWmulti)

Eksamenstid: 4 timer (0900-1300)

Antall eksamensoppgaver: 6

Antall sider (medregnet denne): 9

Antall vedlegg: Ingen

Tillatte hjelpemiddel: Ingen

Lærere: Lars-Petter Helland (928 28 046) lph@hvl.no
 Erlend Raa Vågset (57 72 26 08) erlend.raa.vagset@hvl.no
 Bjarte Kileng (909 97 348) bki@hvl.no

LYKKE TIL!

Oppgave 1 (10% ~ 24 minutter) – Strømmer

Flervalgsoppgave: Oppgavetekst i Wiseflow

Oppgave 2 (10% ~ 24 minutter) – Funksjonelle kontrakter

Flervalgsoppgave: Oppgavetekst i Wiseflow

Oppgave 3 (15% ~ 36 minutter) – Tråder

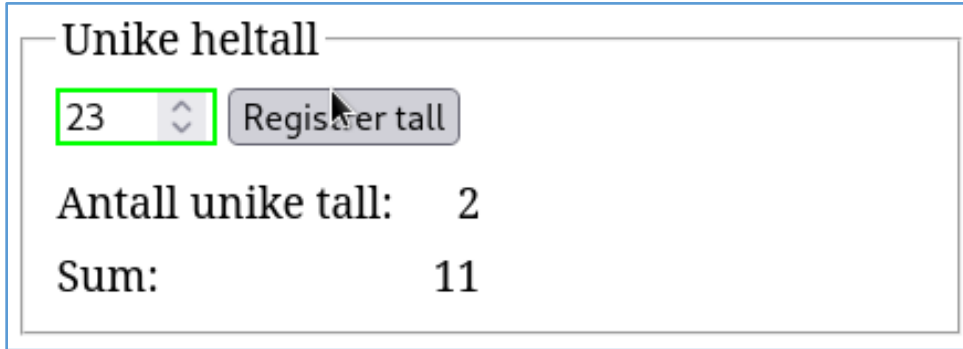
Oppgavetekst i Wiseflow

Oppgave 4 (10% ~ 24 minutter) – JavaScript teori

Flervalgsoppgave: Oppgavetekst i Wiseflow

Oppgave 5 (15% ~ 36 minutter) – JavaScript programmering

En webside inneholder et felt som lar bruker registrere unike heltall, og som viser antallet med unike tall og summen av dem. Skjermbildet under viser hvordan websiden kan se ut:



Skjermbilde 1: Felt for å registrere heltall

I skjermbildet over har bruker allerede registrert tallene 4 og 7, og holder nå på med å registrere tallet 23.

Applikasjonen bruker HTML-koden under for å lese inn tallene:

```
<fieldset id="root">
  <legend>Unike heltall</legend>

  <div>
    <input type="number" size="5">
    <button type="button">Registrer tall</button>
  </div>
  <div class="resultat">
    Antall unike tall: <span>0</span>
    Sum: <span>0</span>
  </div>
</fieldset>
```

Kode-eksempel 1: HTML-kode for å lese inn heltall

Du kan anta at applikasjonen kun vil lese heltall, da alt annet vil sette input-feltet sin tilstand til *invalid*. Dette gjøres automatisk av nettleser. Når feltet sin tilstand er *invalid* endres rammefargen til rød. Dette gjøres av CSS-kode. Dere skal ikke skrive CSS i denne oppgaven!

Input-feltet sin tilstand skal settes til *invalid* også hvis bruker forsøker å registrere et tall som allerede er registrert. Dette må håndteres av JavaScript-koden. Dette er illustrert i skjermbildet under. Observer at dere ikke skal skrive CSS-koden som gjør rammen rød, kun settet input-feltet sin tilstand til *invalid*.



Skjerm bilde 2: Forsøk på å registrere et tall som allerede er registrert

Applikasjonen inneholder følgende JavaScript-kode:

```
class Talldata {
  #tallinput

  // Legg til her eventuelle flere private felt

  constructor(root) {
    this.#tallinput =
      root.getElementsByTagName("input")[0];
    const bt =
      root.getElementsByTagName("button")[0];

    bt.addEventListener("click",
      (event) => { this.#lestall(event) }
    );

    // Legg inn her mer kode hvis nødvendig
  }

  #lestall(event) {
    // Legg inn kode her
  }

  // Legg til eventuelle flere private metoder
}

const rootelement = document.getElementById("root");
new Talldata(rootelement);
```

Oppgave: Fyll inn den manglende koden i JavaScript-klassen **Talldata**.

(Hjelp: Se neste side)

Hjelp:

- Kode-eksempelet under setter tilstanden til *invalid* for et input-element og viser feilmeldingen «Noe er feil»:

```
const input = document.getElementsByTagName("input")[0];
input.setCustomValidity("Noe er feil");
input.reportValidity();
```

- Kode-eksempelet under setter tilstanden til *valid* for et input-element og fjerner feilmeldingen:

```
const input = document.getElementsByTagName("input")[0];
input.setCustomValidity("");
input.reportValidity();
```

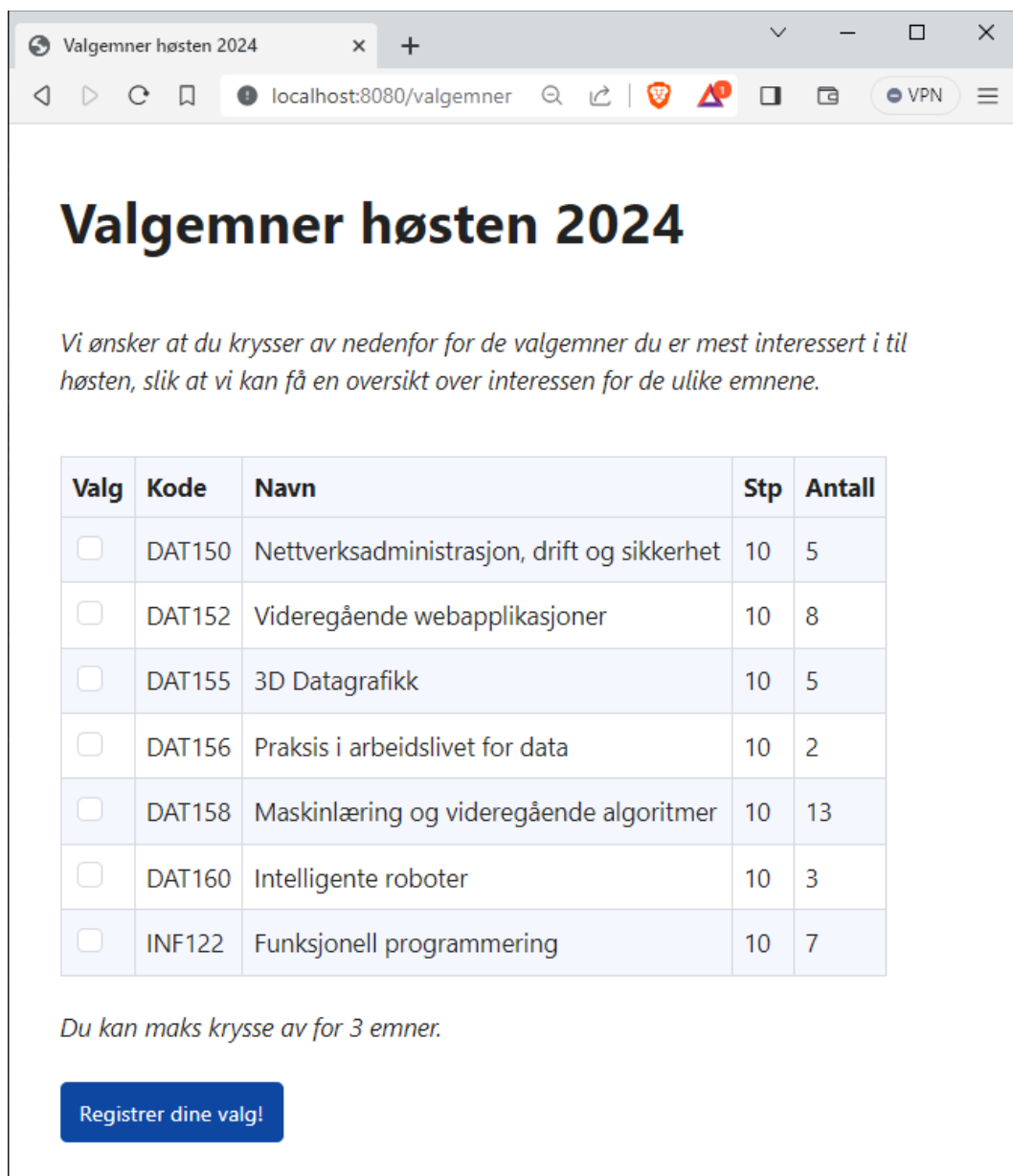
- Både **Map**, **Set** og **Array** har en metode *forEach*.
- **Map** og **Set** sin metode *values* returnerer en iterator over alle verdier.
- **Map** sin metode *keys* returnerer en iterator over alle nøkler.
- **Array** sin statiske metode *from*, og også spre (*spread*) operator kan kopiere et itererbart objekt til en **Array**.
- **Map** har bla. metoder *has*, *set*, *get* og *delete*.
- **Set** har bla. metoder *has*, *add* og *delete*.

Oppgave 6 (40% ~ 96 minutter) – Web backend med Spring MVC

I denne oppgaven skal du jobbe litt med en liten web-applikasjon for å registrere interessen for ulike valgemner for høstsemesteret i 3. klasse.

Applikasjonen har kun én side (se bildet nedenfor) som brukes både til å registrere interesse, og til å se resultatet så langt.

Bruker registrerer interesse for emne(r) ved å huke av i én eller flere checkbox-er til venstre i tabellen og deretter trykke på knappen «Registrer dine valg!». Resultatet fra tidligere innsendte skjemaer vises til høyre i tabellen.



Valgemner høsten 2024

localhost:8080/valgemner

Valgemner høsten 2024

Vi ønsker at du krysser av nedenfor for de valgemner du er mest interessert i til høsten, slik at vi kan få en oversikt over interessen for de ulike emnene.

Valg	Kode	Navn	Stp	Antall
☐	DAT150	Nettverksadministrasjon, drift og sikkerhet	10	5
☐	DAT152	Videregående webapplikasjoner	10	8
☐	DAT155	3D Datagrafikk	10	5
☐	DAT156	Praksis i arbeidslivet for data	10	2
☐	DAT158	Maskinlæring og videregående algoritmer	10	13
☐	DAT160	Intelligente roboter	10	3
☐	INF122	Funksjonell programmering	10	7

Du kan maks krysse av for 3 emner.

Registrer dine valg!

Validering, feilmelding og kvittering

Brukerinput skal valideres, dvs. at det er valgt 1-3 emner, og at alle valgte emner finnes i databasen og går i høstsemesteret.

Ved feil skal en generell feilmelding vises med rødt i toppen av siden, slik:



... (resten av siden er utelatt)

Etter gyldig innsending og registrering vises en kvittering med grønt i toppen av siden, slik:



... (resten av siden er utelatt)

Data i applikasjonen

Data om emner og interessen for disse er lagret i én enkelt tabell i en database, slik:

```
CREATE TABLE emne (  
  kode CHAR(6) PRIMARY KEY,      -- f.eks. 'DAT100'  
  navn VARCHAR,                  -- f.eks. 'Grunnleggende programmering'  
  studiepoeng INTEGER,           -- f.eks. 10  
  semester CHAR(1),              -- 'v'|'h'|'b' for vår, høst eller begge  
  type CHAR(1),                  -- 'o'|'v' for obligatorisk eller valgfag  
  antall INTEGER                 -- antall interesserte  
);
```

Det er definert en tilsvarende entitetsklasse i Java, slik:

```
@Entity
public class Emne {

    @Id private String kode;
    private String navn;
    private Integer studiepoeng;
    private String semester;
    private String type;
    private Integer antall;

    public void registrerInteresse() { antall++; }

    // + gettere for alle egenskaper
}
```

Lagdeling og komponenter i applikasjonen

Datatilgang gjøres via et

```
interface EmneRepository extends JpaRepository<Emne, String>
```

der de interessante metodene for oss er

```
List<Emne> findAll()
Emne findById(String emnekode)
```

Mellom controller og repository skal det legges en

```
@Service class EmneRepoService
```

med metodene

```
List<Emne> finnValgemnerForSemester(String semester) // "v" eller "h"
void registrerInteresseFor(List<String> emnekode)
```

Controlleren

```
@Controller class EmnelisteController
```

skal håndtere følgende requests

```
@GetMapping("/valgemner") // Request om å se nettsiden. Samarbeider
                          // med viewet kalt "emneliste".

@PostMapping("/registrer") // Request om å registrere interesse for
                           // emner. Etterpå vises emnelisten igjen.
```

Arkitekturkrav ellers

- Løsningen skal utformes i Spring MVC
- Applikasjonen skal bruke Model-View-Controller (MVC)
- View-komponenter skal skrives med Java ServerPages (JSP)
- JSP skal bruke Expression Language (EL) og Standard Tag Library (JSTL)
- Det skal benyttes Post-Redirect-Get ved POST-requests

Oppgaver

a – 10%)

Skriv koden for `emneliste.jsp`.

- Det er tilstrekkelig å kun skrive `<body>`-taggen og innholdet i denne.
- Øverst på siden skal det kunne vises feilmelding/kvittering, om aktuelt.
- Det er ikke nødvendig å sette opp emnelisten i en `<table>`. Det gir full score å sette den opp med å bruke en `<p></p>` per linje.
- Tips: `<input type="checkbox" ... >` vil ha samme navn, men ulik verdi for de ulike valgene
- En del elementer i JSP-en har direkte sammenheng med hvordan ting gjøres i controlleren, og det er viktig at det er samsvar mellom JSP og controller (neste spørsmål).

b – 15%)

Skriv koden for `EmnelisteController`, inkl. metoder for GET- og POST-mappingene som er nevnt i oppgaveteksten over, samt en hjelpemetode for validering av input.

- Pass på at løsningen samsvarer med JSP-en (forrige spørsmål).
- Ved innsending av skjema:
 - Request-parametre for checkboxen kommer inn som en List av verdier.
 - Ved ugyldig input skal det ikke registreres noe i databasen, men lages til en generell feilmelding som deretter vises på siden.
 - Ved gyldig input skal valgene registreres i databasen, og det skal lages en kvitteringsmelding som deretter vises på siden.
- Metoden for validering av input skal ha denne signaturen:

```
boolean erGyldigEmnevalg(List<String> emnevalg)
```

Gyldig liste av emnevalg er en liste av 1-3 emnekoder som alle er registrerte høstvalgfag i databasen. Tips for å sjekke match med databasedata: Du kan spørre en List om den `containsAll` elementer i en annen List.

c – 10%)

Skriv koden for `EmneRepoService`

- `List<Emne> finnValgemnerForSemester(String semester)` skal returnere alle valgemner (type 'v') for gitt semester. Emnelisten skal være sortert på emnekode. Du får pluss for å bruke streams i løsningen.
- `void registrerInteresseFor(List<String> emnekoder)` skal oppdatere databasen med registrert interesse for emnene for gitt liste av emnekoder.

d – 5%)

Vi ønsker å enhetsteste metoden for inputvalidering `erGyldigEmnevalg(List<String> emnevalg)`. Skriv en klasse `InputvalideringTest` for denne enhetstesting. Tips: Du kan bruke mockito-rammeverket for å erstatte `EmneRepoService`-en som gir deg databasedata å sammenligne med. Du kan anta at `Emne`-klassen har en parametrisk konstruktør for enkel opprettelse av emne-objekter.

EKSAMENSOPPGÅVE

Nynorsk

Emnekode: DAT108

Emnenamn: Programmering og webapplikasjoner

Klasse: 2. klasse DATA / INF

Dato: 18. desember 2023

Eksamensform: Skriftleg skuleeksamen (Wiseflow | FLOWlock)

Eksamenstid: 4 timer (0900-1300)

Tal på eksamensoppgåver: 6

Tal på sider (medrekna denne): 9

Tal på vedlegg: Ingen

Tillate hjelpemiddel: Ingen

Lærarar: Lars-Petter Helland (928 28 046) lph@hvl.no
 Erlend Raa Vågset (57 72 26 08) erlend.raa.vagset@hvl.no
 Bjarte Kileng (909 97 348) bki@hvl.no

LYKKE TIL!

Oppgave 1 (10% ~ 24 minutt) – Straumar

Fleirvalsoppgave: Oppgåvetekst i Wiseflow

Oppgave 2 (10% ~ 24 minutt) – Funksjonelle kontraktar

Fleirvalsoppgave: Oppgåvetekst i Wiseflow

Oppgave 3 (15% ~ 36 minutt) – Trådar

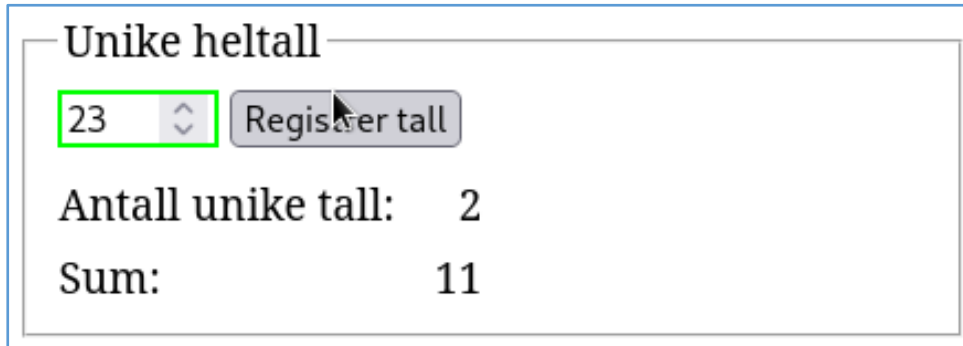
Oppgåvetekst i Wiseflow

Oppgave 4 (10% ~ 24 minutt) – JavaScript teori

Fleirvalsoppgave: Oppgåvetekst i Wiseflow

Oppgave 5 (15% ~ 36 minutt) – JavaScript programmering

Ein webside inneheld eit felt som lar bruker registrere unike heiltal, og som viser talet med unike tal og summen av dei. Skjermbildet under viser korleis websida kan sjå ut:



Skjermbilete 1: Felt for å registrere heiltal

I skjermbildet over har bruker allereie registrert tala 4 og 7, og held nå på med å registrere talet 23.

Applikasjonen bruker HTML-koden under for å lese inn tala:

```
<fieldset id="root">
  <legend>Unike heltall</legend>

  <div>
    <input type="number" size="5">
    <button type="button">Registrer tall</button>
  </div>
  <div class="resultat">
    Antall unike tall: <span>0</span>
    Sum: <span>0</span>
  </div>
</fieldset>
```

Kode-døme 1: HTML-kode for å lese inn heiltal

Du kan anta at applikasjonen berre vil lese heiltal, då alt anna vil sette input-feltet sin tilstand til *invalid*. Dette blir gjort av nettlesar automatisk. Når feltet sin tilstand er *invalid* blir rammefargen endra til rødt. Dette blir gjort av CSS-kode. De skal ikkje skrive CSS i denne oppgåva!

Input-feltet sin tilstand skal bli satt til *invalid* også om bruker forsøker å registrere eit tal som allereie er registrert. Då må JavaScript-kod sette tilstanden til *invalid*. Dette er illustrert i skjermbildet under. Observer at de ikkje skal skrive CSS-kod som gjer ramma rødt, berre settet input-feltet sin tilstand til *invalid*.

Unike heltall

Tallet er allerede registrert

Sum. 11

Skjermbilete 2: Forsøk på å registrere eit tall som allereie er registrert

Applikasjonen inneheld følgjande JavaScript-kode:

```
class Talldata {
  #tallinput

  // Legg til her eventuelle flere private felt

  constructor(root) {
    this.#tallinput =
      root.getElementsByTagName("input")[0];
    const bt =
      root.getElementsByTagName("button")[0];

    bt.addEventListener("click",
      (event) => { this.#lestall(event) }
    );

    // Legg inn her mer kode hvis nødvendig
  }

  #lestall(event) {
    // Legg inn kode her
  }

  // Legg til eventuelle flere private metoder
}

const rootelement = document.getElementById("root");
new Talldata(rootelement);
```

Oppgåve: Fyll inn kode som manglar i JavaScript-klassen **Talldata**.

(Hjelp: Sjå neste side)

Hjelp:

- Dømet med kode under setter tilstanden til *invalid* for et input element og viser feilmeldinga «Noe er feil»:

```
const input = document.getElementsByTagName("input")[0];
input.setCustomValidity("Noe er feil");
input.reportValidity();
```

- Døme med kode under setter tilstanden til *valid* for et input element og fjernar feilmeldinga:

```
const input = document.getElementsByTagName("input")[0];
input.setCustomValidity("");
input.reportValidity();
```

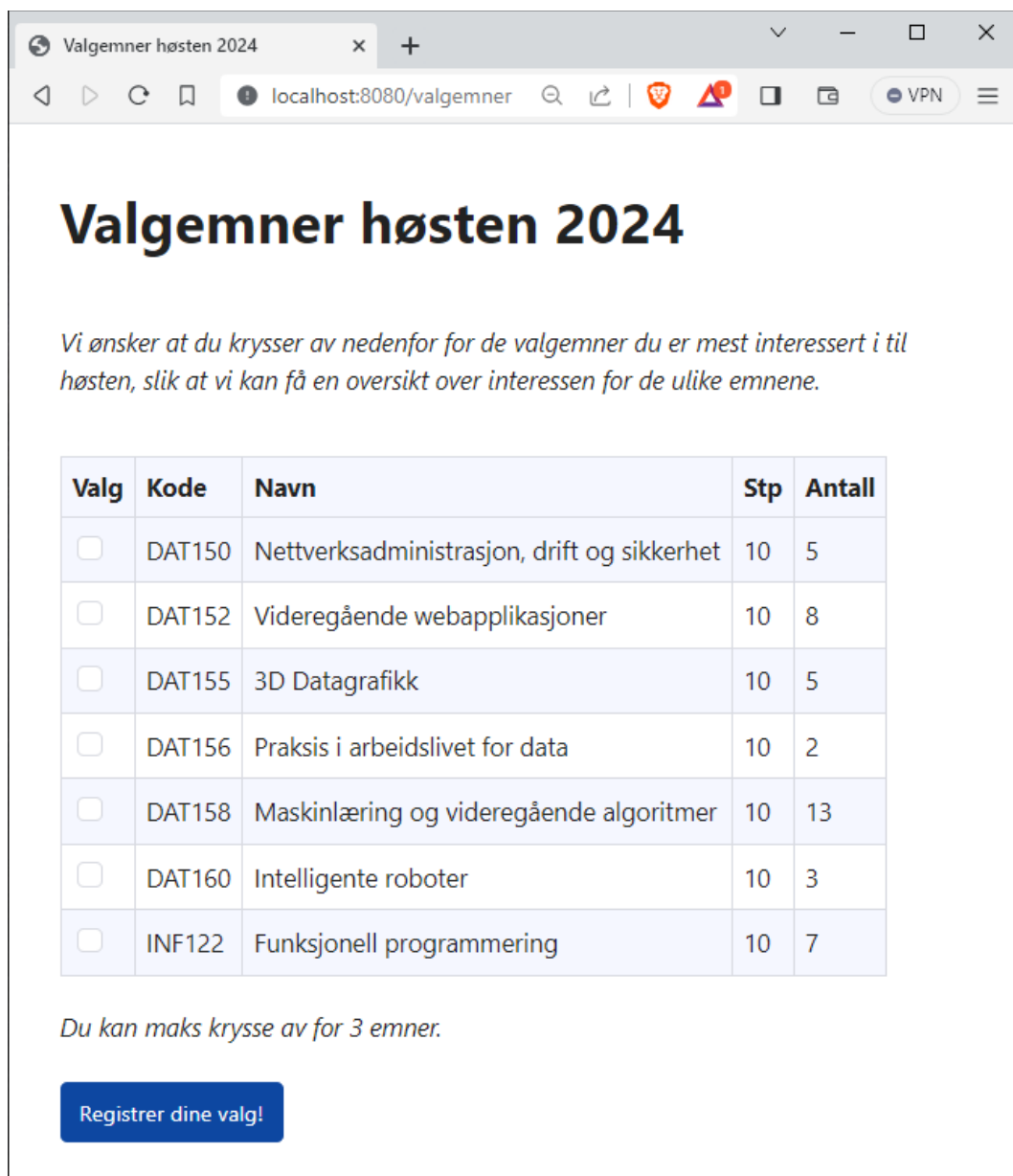
- Både **Map**, **Set** og **Array** har ein metode *forEach*.
- **Map** og **Set** sin metode *values* returnerer ein iterator over alle verdi.
- **Map** sin metode *keys* returnerer ein iterator over alle nøklar.
- **Array** sin statiske metode *from*, og også spreie (*spread*) operator kan kopiere eit itererbart objekt til ein **Array**.
- **Map** har bla. metodar *has*, *set*, *get* og *delete*.
- **Set** har bla. metodar *has*, *add* og *delete*.

Oppgave 6 (40% ~ 96 minutt) – Web backend med Spring MVC

I denne oppgave skal du jobbe litt med ein liten web-applikasjon for å registrere interessa for ulike valemne for haustsemesteret i 3. klasse.

Applikasjonen har berre éi side (sjå biletet nedanfor) som blir brukt både til å registrere interesse, og til å sjå resultatet så langt.

Bruker registrerer interesse for emne(r) ved å huke av i éin eller fleire checkbox-er til venstre i tabellen og deretter trykkje på knappen «Registrer dine valg!». Resultatet frå tidlegare innsende skjema blir vist til høgre i tabellen.



Valgemner høsten 2024

localhost:8080/valgemner

Valgemner høsten 2024

Vi ønsker at du krysser av nedenfor for de valgemner du er mest interessert i til høsten, slik at vi kan få en oversikt over interessen for de ulike emnene.

Valg	Kode	Navn	Stp	Antall
☐	DAT150	Nettverksadministrasjon, drift og sikkerhet	10	5
☐	DAT152	Videregående webapplikasjoner	10	8
☐	DAT155	3D Datagrafikk	10	5
☐	DAT156	Praksis i arbeidslivet for data	10	2
☐	DAT158	Maskinlæring og videregående algoritmer	10	13
☐	DAT160	Intelligente roboter	10	3
☐	INF122	Funksjonell programmering	10	7

Du kan maks krysse av for 3 emner.

Registrer dine valg!

Validering, feilmelding og kvittering

Brukarinput skal validerast, dvs. at det er valt 1-3 emnar, og at alle valde emne finst i databasen og går i haustsemesteret.

Ved feil skal ei generell feilmelding visast med raudt i toppen av sida, slik:



... (resten av sida er utelate)

Etter gyldig innsending og registrering vises en kvittering med grønt i toppen av sida, slik:



... (resten av sida er utelate)

Data i applikasjonen

Data om emne og interessa for desse er lagra i éin enkelt tabell i ein database, slik:

```
CREATE TABLE emne (  
  kode CHAR(6) PRIMARY KEY,      -- f.eks. 'DAT100'  
  navn VARCHAR,                  -- f.eks. 'Grunnleggende programmering'  
  studiepoeng INTEGER,           -- f.eks. 10  
  semester CHAR(1),              -- 'v'|'h'|'b' for vår, høst eller begge  
  type CHAR(1),                  -- 'o'|'v' for obligatorisk eller valgfag  
  antall INTEGER                 -- antall interesserte  
);
```

Det er definert en tilsvarende entitetsklasse i Java, slik:

```
@Entity
public class Emne {

    @Id private String kode;
    private String navn;
    private Integer studiepoeng;
    private String semester;
    private String type;
    private Integer antall;

    public void registrerInteresse() { antall++; }

    // + gettere for alle egenskaper
}
```

Lagdeling og komponentar i applikasjonen

Datatilgang blir gjort via et

```
interface EmneRepository extends JpaRepository<Emne, String>
```

der de interessante metodane for oss er

```
List<Emne> findAll()
Emne findById(String emnekode)
```

Mellom controller og repository skal det leggjast ein

```
@Service class EmneRepoService
```

med metodane

```
List<Emne> finnValgemnerForSemester(String semester) // "v" eller "h"
void registrerInteresseFor(List<String> emnekoder)
```

Controlleren

```
@Controller class EmnelisteController
```

skal handtere følgjande requests

```
@GetMapping("/valgemner") // Request om å se nettsiden. Samarbeider
                           // med viewet kalt "emneliste".

@PostMapping("/registrer") // Request om å registrere interesse for
                           // emner. Etterpå vises emnelisten igjen.
```

Arkitekturkrav elles

- Løysinga skal utformast i Spring MVC
- Applikasjonen skal bruke Model-View-Controller (MVC)
- View-komponentar skal skrivast med Java ServerPages (JSP)
- JSP skal bruke Expression Language (EL) og Standard Tag Library (JSTL)
- Det skal nyttast Post-Redirect-Get ved POST-requests

Oppgåver

a – 10%)

Skriv koden for `emneliste.jsp`.

- Det er tilstrekkeleg å berre skrive `<body>`-taggen og innhaldet i denne.
- Øvst på sida skal det kunne visast feilmelding/kvittering, om aktuelt.
- Det er ikkje nødvendig å setje opp emnelista i ein `<table>`. Det gir full score å setje den opp med å bruke ein `<p></p>` per linje.
- Tips: `<input type="checkbox" ... >` vil ha same namn, men ulik verdi for de ulike vala
- Ein del element i JSP-en har direkte samanheng med korleis ting blir gjorde i controlleren, og det er viktig at det er samsvar mellom JSP og controller (neste spørsmål).

b – 15%)

Skriv koden for `EmnelisteController`, inkl. metodar for GET- og POST-mappingene som er nemnt i oppgåveteksten over, og dessutan ein hjelpemetode for validering av input.

- Pass på at løysinga samsvarer med JSP-en (førre spørsmål).
- Ved innsending av skjema:
 - Request-parametrar for checkboxen kjem inn som ein List av verdier.
 - Ved ugyldig input skal det ikkje registrerast noko i databasen, men lagast til ei generell feilmelding som deretter vises på sida.
 - Ved gyldig input skal vala registrerast i databasen, og det skal lagast ei kvitteringsmelding som deretter blir vist på sida.
- Metoden for validering av input skal ha denne signaturen:
`boolean erGyldigEmnevalg(List<String> emneval)`
Gyldig liste av emneval er en liste av 1-3 emnekodar som alle er registrerte haustvalfag i databasen. Tips for å sjekke match med databasedata: Du kan spørje ei List om den containsAll element i ei annan List.

c – 10%)

Skriv koden for `EmneRepoService`

- `List<Emne> finnValgemnerForSemester(String semester)` skal returnere alle valemne (type 'v') for gitt semester. Emnelista skal være sortert på emnekode. Du får pluss for å bruke streams i løysninga.
- `void registrerInteresseFor(List<String> emnekoder)` skal oppdatere databasen med registrert interesse for emna for gitt liste av emnekodar.

d – 5%)

Vi ønskjer å einingsteste metoden for inputvalidering `erGyldigEmnevalg(List<String> emneval)`. Skriv ein klasse `InputvalideringTest` for denne einingstestinga. Tips: Du kan bruke mockito-rammeverket for å erstatte `EmneRepoService`-en som gir deg databasedata å samanlikne med. Du kan rekne med at `Emne`-klassen har ein parametrisk konstruktør for enkel oppretting av emne-objekt.